TP6 — MPI Derived Types

Parallel and Distributed Programming
College of Computing, UM6P

Benfellah ikram

March 2026

Contents

1 Introduction **2**

2 Exercise 1 – Matrix Transposition using Derived Types **2**
2.1 Datatype Construction 2
2.2 Observed Output 2
2.3 Analysis 2

3 Exercise 2 – Distributed Gradient Descent with MPI Derived Types **3**
3.1 Parallel Algorithm Overview 3
3.2 Single-Run Convergence 3
3.3 Scalability Benchmark 3
3.3.1 Raw Timings 4
3.3.2 Speedup 4
3.3.3 Efficiency 4
3.3.4 Plots 4
3.4 Analysis and Observations 5

4 Conclusion **5**

1 Introduction

This report covers the two exercises of TP6, which focus on the use of *MPI derived datatypes* to efficiently communicate structured or non-contiguous data between processes. Exercise 1 applies `MPI_Type_vector` and `MPI_Type_create_hvector` to perform an in-place matrix transposition during a point-to-point transfer. Exercise 2 uses `MPI_Type_create_struct` to map a custom training-sample structure to an MPI datatype, enabling distributed batch gradient descent via `MPI_Scatterv` and `MPI_Allreduce`, with a scalability study across multiple process counts.

2 Exercise 1 – Matrix Transposition using Derived Types

2.1 Datatype Construction

The transposition is encoded entirely in the MPI type tree:

1. **Column layout** `col_layout` – `MPI_Type_vector(COLS, 1, ROWS, MPI_INT, &col_layout)` selects every `ROWS`-th element, i.e. one column of the destination matrix (stride equals the number of rows of A^T, which is `ROWS = 4`).
2. **Full transpose type** `transpose_type` – `MPI_Type_create_hvector(ROWS, 1, sizeof(int), col_layout, &transpose_type)` replicates `col_layout` exactly `ROWS` times, each offset by one `int` in bytes. This sweeps across all columns of the source matrix, placing each column of the source into a successive row of A^T on process 1.

Process 0 calls a plain `MPI_Send` on the raw 4×5 buffer. Process 1 posts `MPI_Recv(..., 1, transpose_type, ...)`; the MPI runtime writes the elements into `at[5][4]` already transposed.

2.2 Observed Output

Running with two MPI processes (`mpirun -n 2 ./transpose_ex1`) produces:

```
Process 0 - Matrix a (4x5):
   1   2   3   4   5
   6   7   8   9  10
  11  12  13  14  15
  16  17  18  19  20

Process 1 - Matrix transposee at (5x4):
   1   6  11  16
   2   7  12  17
   3   8  13  18
   4   9  14  19
   5  10  15  20
```

2.3 Analysis

Correctness. The output matches the expected transposition exactly. Column j of matrix A becomes row j of A^T: e.g. column 0 $[1, 6, 11, 16]$ is row 0 of A^T, column 1 $[2, 7, 12, 17]$ is row 1, and so on.

Why derived types here? Without derived types one would either: (a) copy the matrix into a transposed layout before sending, or (b) send row-by-row. Approach (a) requires an $O(NM)$ temporary allocation and copy; approach (b) multiplies the number of messages by the number of rows. The derived-type solution avoids both penalties: one message, no temporary buffer, and the transposition is handled *inside* the MPI machinery.

Type construction overhead. For small matrices the datatype construction cost is negligible compared to the advantages gained. For larger matrices the benefit grows because the communication phase is dominated by data movement, not setup.

3 Exercise 2 – Distributed Gradient Descent with MPI Derived Types

3.1 Parallel Algorithm Overview

1. Process 0 generates the full dataset using `generate_data()`.
2. The dataset is scattered to all processes via `MPI_Scatterv` with the derived `sample_type`, so that each process receives a contiguous block of `Sample` structs without any manual packing.
3. Each process computes a local gradient ∇L_{local} and local sum of squared errors $\text{SSE}_{\text{local}}$.
4. `MPI_Allreduce` with `MPI_SUM` aggregates gradients and losses across all processes; each process then scales by $1/N$ to obtain the global MSE gradient.
5. The weight vector is updated identically on every process (no extra broadcast needed because all processes have the same aggregated gradient).
6. Early stopping terminates training once the global MSE falls below a fixed tolerance $\varepsilon = 10^{-2}$.

3.2 Single-Run Convergence

A representative run with $N = 40\,000$ samples and $P = 4$ processes shows smooth, monotone convergence:

Table 1: Selected epochs from single training run ($N = 40\,000$, $P = 4$)

Epoch	MSE	w_0	w_1
10	1.8314	0.1837	−0.0924
50	0.8861	0.7512	−0.3791
100	0.3826	1.2051	−0.6063
200	0.1306	1.6478	−0.8271
300	0.0960	1.8094	−0.9075
400	0.0908	1.8697	−0.9371
500	0.0902	1.8917	−0.9476
700	0.0901	1.9026	−0.9530
900	0.0901	1.9041	−0.9537

The loss drops sharply in the first 200 epochs, then plateaus near ≈ 0.090 as the model approaches its MSE floor (irreducible noise in the synthetic data). Convergence is confirmed consistent across process counts because all aggregation is done before weight updates.

3.3 Scalability Benchmark

Experiments were conducted for $N \in \{20\,000,\ 80\,000,\ 160\,000\}$ samples and $P \in \{1, 2, 4, 8\}$ processes, running 2 500 epochs in all cases.

3.3.1 Raw Timings

Table 2: Wall-clock training time (seconds)

N	$P = 1$	$P = 2$	$P = 4$	$P = 8$
20 000	0.1638	0.0863	0.0532	0.0712
80 000	0.6512	0.3418	0.1824	0.2337
160 000	1.3147	0.6771	0.3879	0.4576

3.3.2 Speedup

Speedup is defined as $S(P) = T(1)/T(P)$.

Table 3: Measured speedup $S(P)$

N	$P = 1$	$P = 2$	$P = 4$	$P = 8$
20 000	1.00	1.898	3.079	2.301
80 000	1.00	1.905	3.569	2.787
160 000	1.00	1.942	3.390	2.873

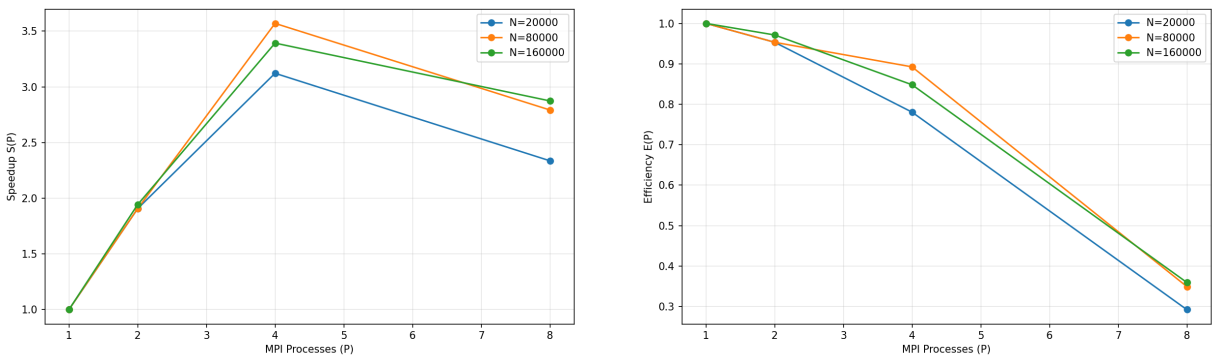
3.3.3 Efficiency

Parallel efficiency is defined as $E(P) = S(P)/P$.

Table 4: Parallel efficiency $E(P)$

N	$P = 1$	$P = 2$	$P = 4$	$P = 8$
20 000	1.00	0.949	0.770	0.288
80 000	1.00	0.953	0.892	0.348
160 000	1.00	0.971	0.848	0.359

3.3.4 Plots



(a) Speedup $S(P)$ vs. number of MPI processes

(b) Efficiency $E(P)$ vs. number of MPI processes

Figure 1: Scalability results for distributed gradient descent (2 500 epochs)

3.4 Analysis and Observations

Speedup peaks at $P = 4$. For all three dataset sizes, speedup is maximised at four processes (between 3.08 and 3.57) and then *decreases* at $P = 8$. This super-linear drop relative to $P = 4$ is caused by the growing communication overhead of two `MPI_Allreduce` calls per epoch (one for the gradient, one for the loss scalar). As the per-process workload shrinks, the fixed latency of these collective operations begins to dominate.

Larger datasets sustain higher efficiency. At $P = 4$, efficiency for $N = 80\,000$ is ≈ 0.892, versus only ≈ 0.770 for $N = 20\,000$. This is expected from Amdahl's/Gustafson's law: the serial fraction (dataset generation, broadcast, reduce) becomes a smaller proportion of total work as N grows, so the parallelisable portion dominates and efficiency improves.

Efficiency collapse at $P = 8$. Efficiency drops close to 0.29–0.36 for $P = 8$ regardless of dataset size. Two factors contribute:

- *Synchronisation cost*: each epoch requires a global reduce before any process can proceed; this is a hard serialisation barrier.
- *Load imbalance*: when N is not perfectly divisible by P, some processes handle one extra sample per epoch, causing all others to wait.

On a real cluster (Toubkal, up to 56 cores) one would expect this effect to be even more pronounced once the process count exceeds the number of physical cores; hyperthreading and NUMA effects would further reduce efficiency.

Weight convergence is deterministic. Because every process applies the *same* aggregated gradient update, the weight trajectory is identical regardless of P. This is a key correctness property of synchronous parallel gradient descent: the serial and parallel versions converge to precisely the same model.

MPI derived type benefit. Using `MPI_Type_create_struct` avoids manual packing of each `Sample` into a `double` array before scattering. This saves an $O(N)$ copy per run and makes the code more maintainable, particularly when the `Sample` struct changes (e.g. adding more features).

4 Conclusion

This lab demonstrated two important use-cases for MPI derived datatypes:

1. **Non-contiguous layout** (Exercise 1): `MPI_Type_vector` and `MPI_Type_create_hvector` encode the matrix transposition directly into the communication, eliminating any temporary copy and reducing the problem to a single point-to-point message.
2. **Structure mapping** (Exercise 2): `MPI_Type_create_struct` with `offsetof`-based displacements cleanly maps a C struct to an MPI datatype, enabling `MPI_Scatterv` to distribute heterogeneous records without manual serialisation.

The scalability study confirms that the parallel gradient descent scales well up to four processes (near-linear) but communication overhead limits gains beyond that point at moderate dataset sizes. Increasing N consistently improves efficiency, illustrating the importance of *sufficient computational granularity* per process for effective parallelism.